

Reviewer Pack — Minimal Rank of Hodge Structures Bounds SAT Refutation Tree ...

Entry 6866d4357334 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 06:33:06 UTC

Minimal Rank of Hodge Structures Bounds SAT Refutation Tree Width

Entry ID: 6866d4357334 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 06:33:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: SAT Refutation Tree Width

Statement:

{‘s1’: ‘For any Boolean satisfiability problem (SAT), the minimal rank of its associated Hodge structure is equal to the minimum SAT refutation tree width over all possible refutations.’, ‘s2’: ‘This implies that any refutation with a tree width greater than the minimal Hodge structure rank cannot be polynomially simulated.’, ‘s3’: ‘Conversely, if there exists a refutation with a tree width strictly less than the minimal Hodge structure rank, then it can be transformed into a simpler refutation with the same truth table in polynomial time.’}

Rationale (proposer’s reasoning):

{‘s1’: ‘The Hodge theory of an algebraic variety provides a rich source of geometric and algebraic invariants that could potentially encode complexity-theoretic properties.’, ‘s2’: ‘Exploring such links might reveal new structures that are not apparent from traditional complexity measures, thus providing deeper insights into the nature of computational problems.’, ‘s3’: ‘The Hodge structure of a Boolean satisfiability problem would provide a novel perspective on its complexity, potentially leading to efficient algorithms for SAT solving.’}

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 23137c2f4d6aa1f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal Hodge structure rank and the minimum SAT refutation tree width across all 30 seeds is greater

than or equal to 0.8, with an average rank difference of no more than 3 for at least 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - hodge theory and sat refutation tree width - minimal rank of hodge structures in sat problems - algebraic geometry hodge structure related to complexity theory sat

Top relevant hits considered: - [<http://arxiv.org/abs/1004.0653v2>] Exact Ramsey Theory: Green-Tao numbers and SAT - [<http://arxiv.org/abs/1304.5498v2>] A SAT Approach to Clique-Width - [<http://arxiv.org/abs/1505.03340v2>] HordeSat: A Massively Parallel Portfolio SAT Solver - [<http://arxiv.org/abs/1803.01936v4>] Variations of Hodge Structures of Rank Three k-Higgs Bundles and Moduli Spaces of Holomorphic Triples - [<http://arxiv.org/abs/2406.13557v1>] satsuma: Structure-based Symmetry Breaking in SAT - [<http://arxiv.org/abs/1908.01624v1>] Learned Clause Minimization in Parallel SAT Solvers - [<http://arxiv.org/abs/1912.00347v3>] Equiresidual algebraic geometry I: The affine theory - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_hodge_structure(formula):
        # Simplified Hodge structure computation (placeholder)
        return len(formula)

    def construct_refutation(formula):
        # Simplified refutation construction (placeholder)
        return random.randint(0, 10)
```

```

n = random.choice([5, 10, 15, 20, 30, 40])
formula = generate_boolean_formula(n)
hodge_rank = compute_hodge_structure(formula)
refutation_width = construct_refutation(formula)

return {
    "metric_name": "Hodge Rank vs Refutation Width",
    "metric_value": abs(hodge_rank - refutation_width),
    "instances_tested": 1,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required analysis to confirm or falsify the conjecture. | next: Re-run the test with increased

time limits and ensure that it completes without crashing. If the test passes, re-evaluate the results against the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11920
2	preregistration	ollama_remote	glm4:latest	0	0	6043
3	novelty	ollama_remote	glm4:latest	0	0	4549
4	novelty	ollama_remote	glm4:latest	0	0	9778
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12207
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8804
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7799
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6040
9	judge	ollama_remote	glm4:latest	0	0	22218

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89358 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/6866d4357334.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6866d4357334.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6866d4357334.tar.gz (if generated)